UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad decana de América

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingeniria de Sistemas

Estructura de Datos

Prof. Gilberto A. Salinas

Semana 03
Métodos de Ordenamiento o clasificación

Ciclo 2026-1

Agenda

1. Introducción
2. Definición
3. Clasificación
4. Métodos de ordenamiento interno (MOI)
5. Métodos de ordenamiento externo. (MOE)
6. Resumen
7. Referencias

1. Introducción

1. **Ordenaamiento**
   - Se refiere a la reorganización de los elementos de una estructura de datos según un operador de comparación de los elementos.
     - En algún orden o secuencia especifica para datos numéricos o datos alfanuméricos.
   - El operador de comparación se utiliza para decidir el nuevo orden de los elementos en la respectiva estructura de datos.

1. Introducción

1. Importancia de los algoritmos de Ordenaamiento

- Son importantres porque reduce la complejidad de un problema. Existe una amplia gama de aplicaciones para estos algoritmos, incluidos algoritmos de búsqueda, algoritmos de bases de datos, métodos de dividir y venceras y algoritmos de estructura de datos.

1. Introducción

1. **Importancia de los algoritmos de Ordenaamiento**
   - Cuando se tienen cientos de datos que se desean imprimir, es posible organizarlos de alguna manera.
   - Una vez ordenados los elementoss, podemos obtener el k-ésimo elemento más pequeño y el k-ésimo más grande en tiempo O(1).
   - Buscar cualquier elemento se vuelve fácil.
     - Podemos utilizar el método de búsqueda binaria.
     - Por lo tanto, la ordenación se vuelve importante en este caso.
   - Se pueden utilizar en software y en problemas conceptuales para resolver problemas más avanzados.

1. Introducción

1. Tipos de tecnicas de ordenamiento

Se utilizan varios algoritmos de ordenación en estructura de datos. Veamos:

- **Basados en comparacion.** Se comparan los elementos con un algoritmo de ordenamiento basado en comparacion.

- **No basados en comparacion**. Se comparan los elementos en un algoritmo de ordenacion no basados en comparacion

1. Introducción

1. **Basados en comparacion**
   - Burbuja
   - Insercion
   - Seleccion
   - Quick sort
   - Merge sort
   - Heap sort

1. **Basado en No comparacion**
   - Counting sort
   - Bucket sort
   - Pigeonhole sort
   - Radix sort
   - Comb sort
   - Tim sort

1. Introducción

1. Introducción
   - Es la operación de organizar un conjunto de elementos en algún orden o secuencia especifica para datos numéricos o datos alfanuméricos de tipo de carácter.
   - Las operaciones comunes de ordenamiento son listas de números, de archivos o de nombres en una agenda.
   - Es importante la operación de ordenamiento para optimizar otros algoritmos como los de búsqueda.

2. Definición

El problema de ordenamiento,

- Dados los elementos: E = $v_1, v_2, v_3, \ldots, v_N$
- La ordenación consiste es poner en orden ascendente o descendente con el propósito de clasificarlos para facilitar la manipulación de datos.
  S = $v_{i1}, v_{i2}, v_{i3}, \ldots, v_{iN}$.

ACCION ordenar (Vector x, ENTERO dx)

//Ordena los elementos del vector x en forma ascendente

PRE: x={} $\vee$ x = {e1, e2, e3, …, eN}

POS: “Vector vacioooo” $\vee$ x = {e1 > e2 > e3, >…,> eN}

3. Clasificación

Los métodos de ordenamiento se clasifican en :

- **Ordenación interna.** Es el ordenamiento en memoria principal
  - **Intercambio.** Burbuja, b con señal, b de sacudida, QuickSort. (n^2) (Cairo 2014)
  - **Inserción**. Inserción, Inserción binaria
  - **Selección.** Seleccion, Monton

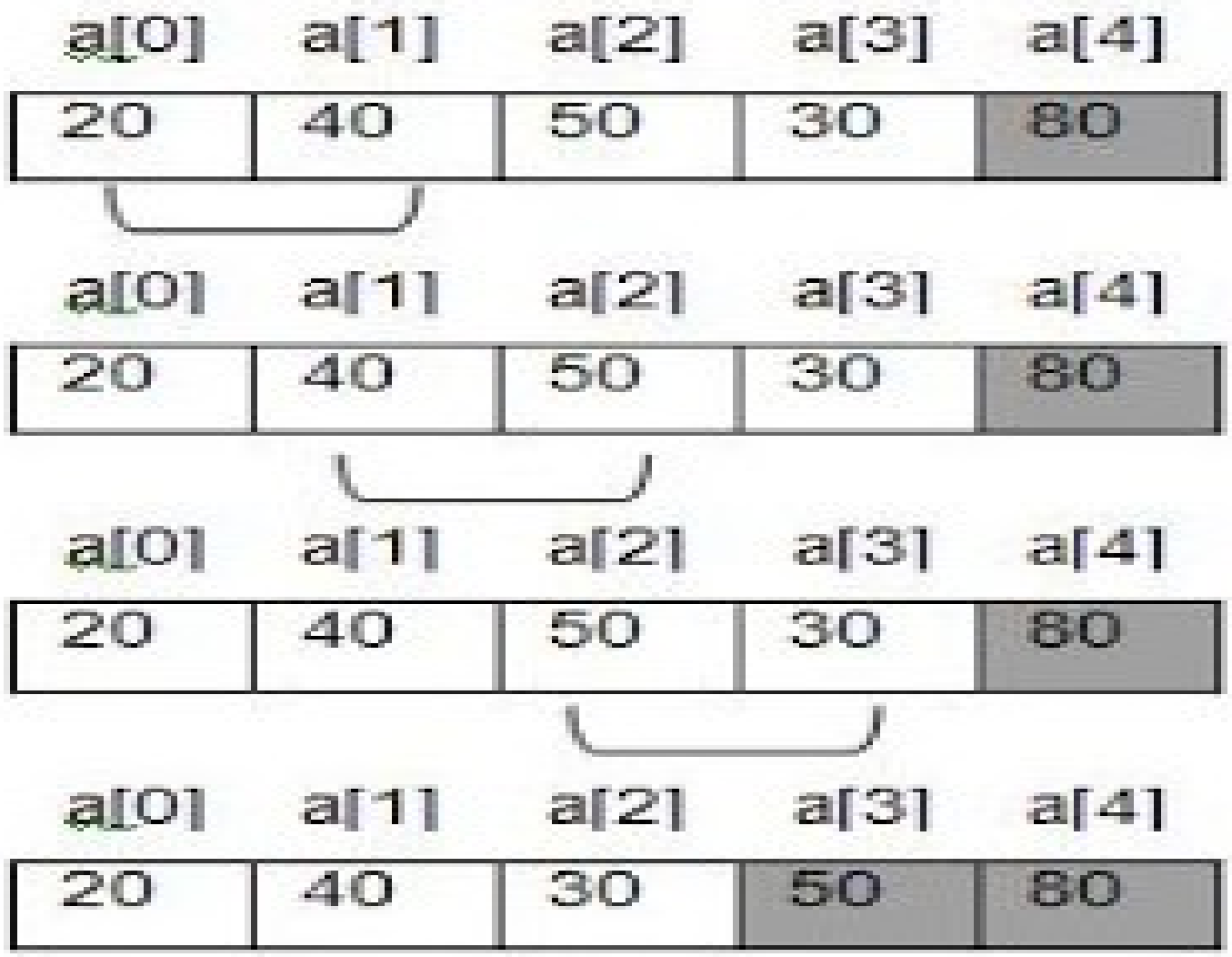
- **Ordenación externa.** Es el ordenamiento de memoria secundaria
  - **Intercalación**
  - **Ordenación**

4. MOI. de intercambio: Burbuja

- Se basa en el principio de comparar pares de elementos adyacentes e intercambiarlos entre si hasta que estén ordenados.

- **Los pasos son:**
  - Comparar x[i] y x[i+1], si están ordenados, se deja como están, en caso contrario se ***intercambian*** entre si.
  - Se avanza una posición ahora se comparan los elementos 2 y 3,de nuevo se intercambian si fuera necesario.
  - El proceso continua hasta que cada uno de los elementos del vector ha sido comparado con los elementos adyacentes y se han realizado los intercambios necesario.
  - Seudo: tmp $\leftarrow$ x[i], x[i] $\leftarrow$ x[i+1], x[i+1] $\leftarrow$ tmp

4. MOI de intercambio: Burbuja



4. MOI de intercambio: Burbuja

0	1	2	3	4
20	40	50	30	80

0	1	2	3	4
20	40	50	30	80

0	1	2	3	4
20	40	50	30	80

0	1	2	3	4
20	40	30	50	80

4. Ventajas y deventajas

- **Ventajas**
  - Fácil implementación
  - No requiere memoria adicional O(1)
- **Desventajas**
  - Muy lento, muchas comparaciones y muchos intercambios
  - Tiene un costo de $O(n^2)$.

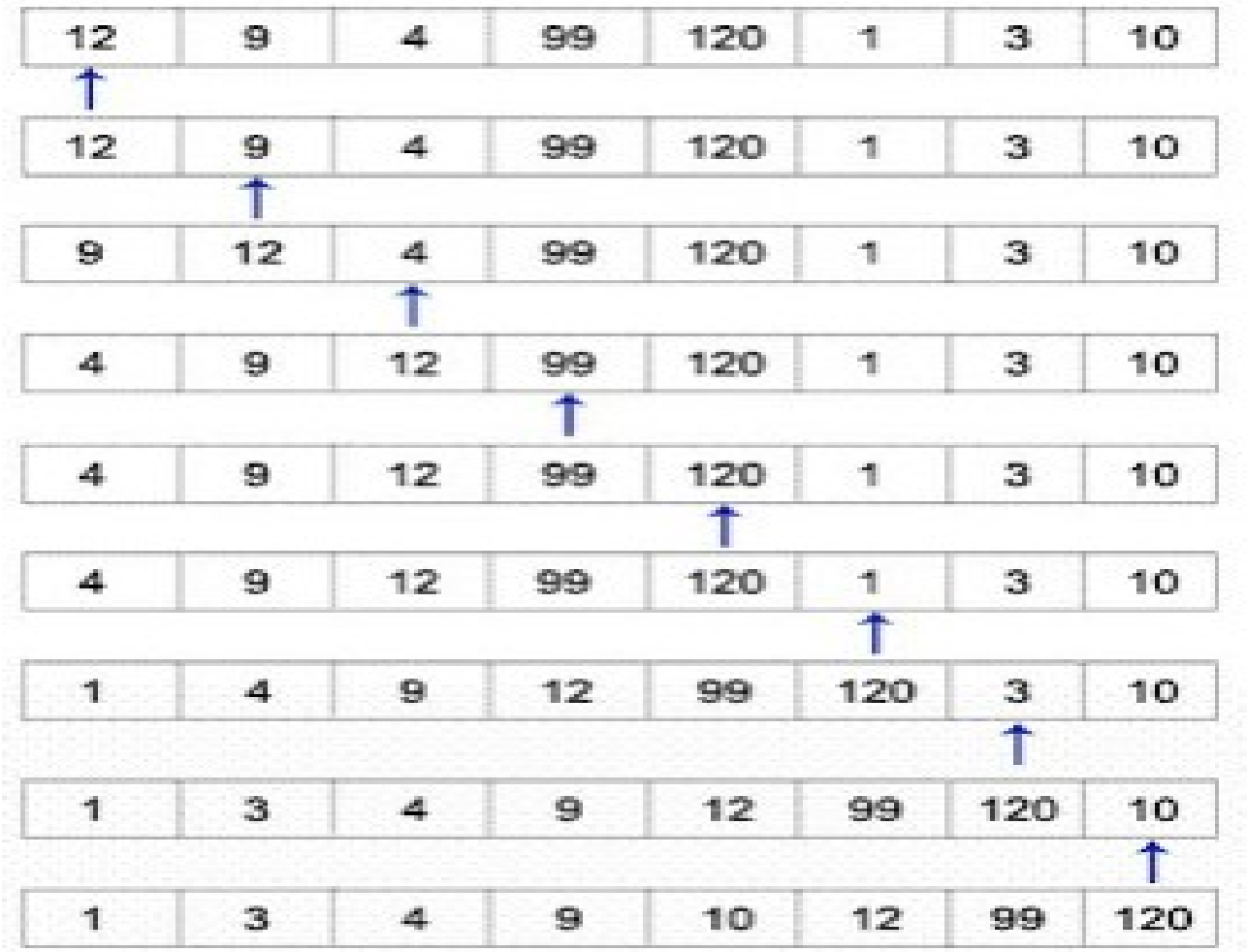
4. MOI: de Inserción

- Se basa en el principio de insertar un elemento en el vector ***en una parte ya ordenada*** del vector y comenzar de nuevo con los elementos restantes, denominado también el método de la baraja.

- **Los pasos son:**
  - El método se basa en comparaciones y desplazamientos sucesivas a una sub lista ya ordenada.
  - El algoritmo de ordenación de un vector se realiza con un recorrido de todo el vector y la inserción del elemento correspondiente en el lugar adecuado.

4. MOI. de Inserción

- Tomado: http://fiec108.blogspot.com/



2. Ventajas y desventajas

- **Ventajas**
  - Fácil implementación.
  - Requerimientos mínimos de memoria O(1).
- **Desventajas**
  - Lento
  - Numerosas comparaciones
  - Tiene un costo de $O(n^2)$ en el peor de los casos

4. MOI: de Selección

- Se basa en el principio de buscar el menor elemento del vector y colocarlo en la primera posición, luego se busca el segundo elemento mas pequeño y se coloca en la segunda posición y así sucesivamente.

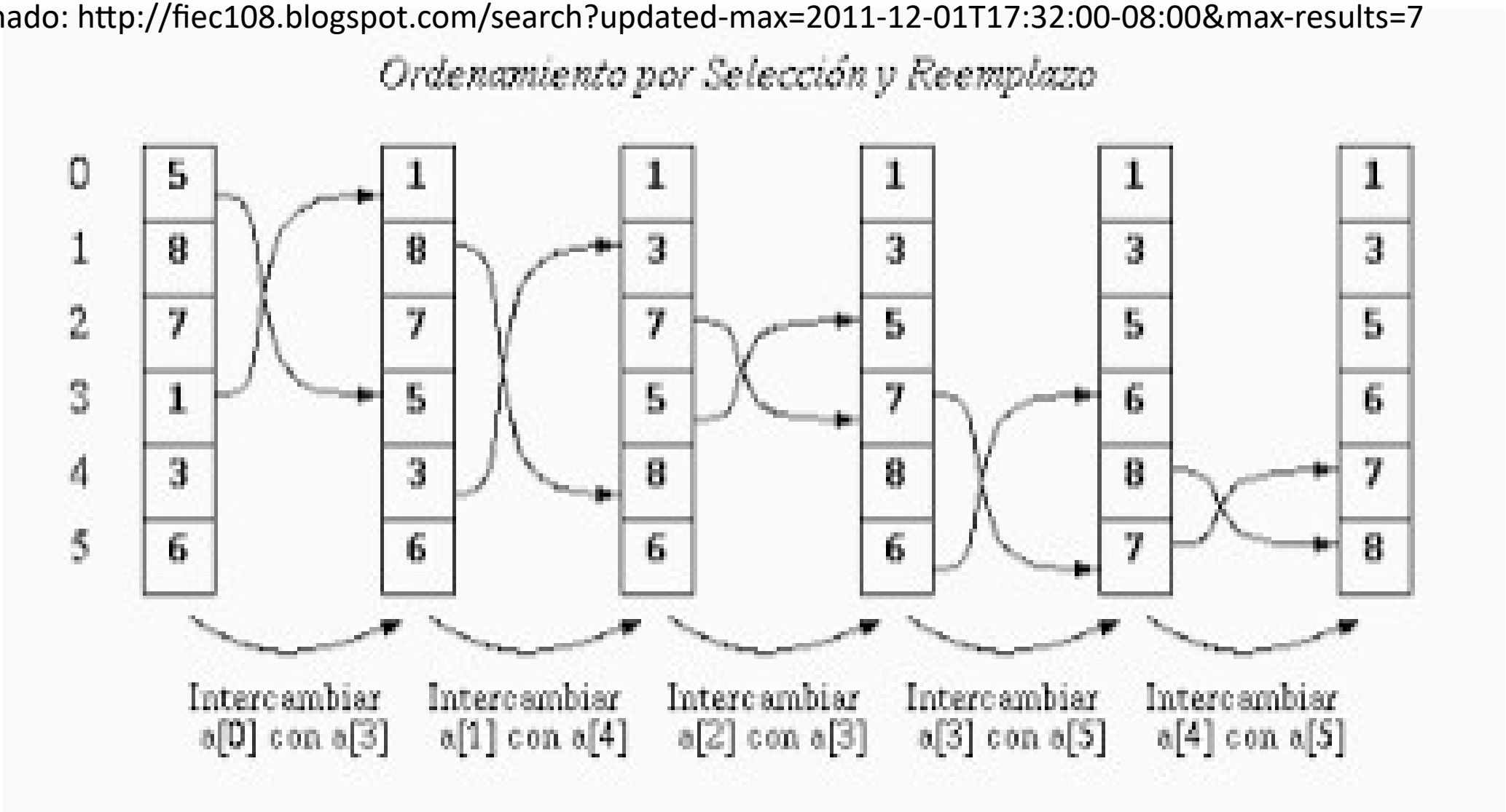
- **Los pasos son:**
  - Seleccionar el menor elemento del vector de n elementos.
  - Intercambiar dicho elemento con el primero.
  - Repetir estas operaciones con los n-1 elementos restantes, seleccionando el segundo elemento, continuar con los n-2 elementos restantes hasta que solo quede el mayor.

4. MOI de Selección

```
[9, 3, 1, 3, 5, 2, 7]
                    [                   9, 3, 1, 3, 5, 2, 7]
                    [1,                 9, 3, 3, 5, 2, 7]
                    [1, 2,              9, 3, 3, 5, 7]
                    [1, 2, 3,           9, 3, 5, 7]
                    [1, 2, 3, 3,        9, 5, 7]
                    [1, 2, 3, 3, 5,     9, 7]
[                   1, 2, 3, 3, 5, 7,   9]
[1, 2, 3, 3, 5, 7, 9]
```

4. MOI de Selección

Tomado: http://fiec108.blogspot.com/search?updated-max=2011-12-01T17:32:00-08:00&max-results=7



4. Ventajas y desventajas

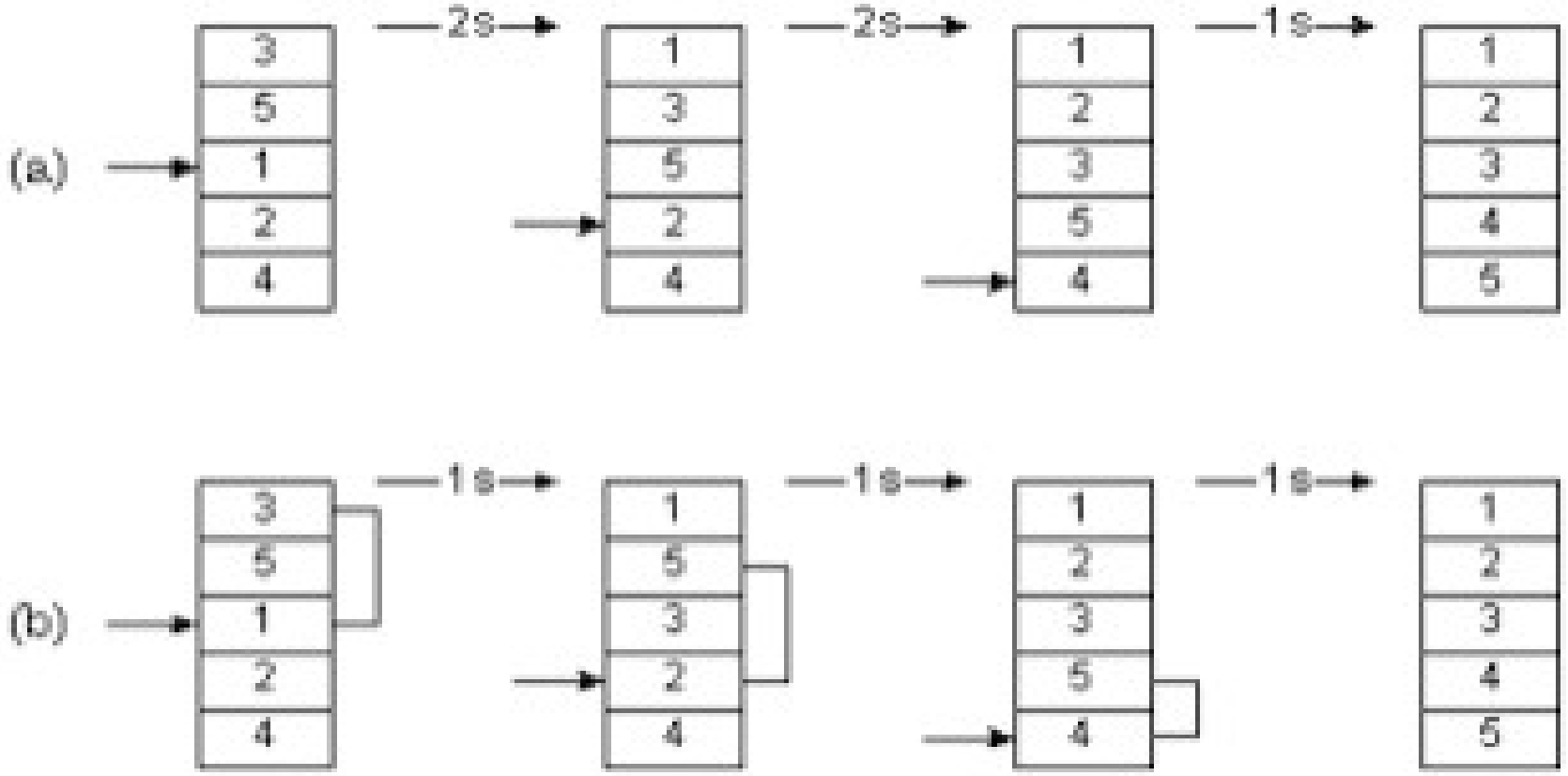
- **Ventajas**
  - Fácil de implementar
  - Realiza pocos intercambios
  - Rendimiento constante, poca diferencia entre el peor y mejor caso.
  - No requiere de memoria adicional O(1)
- **Desventajas**
  - Lento.
  - Realiza numerosas comparaciones
  - Tiene un costo de $O(n^2)$
  - Es inestable

4. MOI: Shell (M. Inserción)

- Se basa en el principio de método inserción directa y es una mejora de esta. También es denominado método de incrementos decrecientes.
- **Los pasos son:**
  - Se modifican los saltos contiguos resultantes de las comparaciones por saltos de mayor tamaño.
  - Generalmente se toma salto inicial n/2 (n dimensión del vector) se comparan los elementos y se intercambian si es necesario.
  - Luego se reduce a la mitad en cada iteración hasta que el salto sea de 1

4. MOI. Shell (M. Inserción)

- Tomado de: http://fiec108.blogspot.com/



2. Ventajas y desventajas

- **Ventajas**
  - Mejor rendimiento que el método de inserción
  - En el mejor de los casos tiene un costo $O(n^{1.25})$
  - No requiere de memoria adicional. O(1)
- **Desventajas**
  - En el peor de los casos tiene un costo de $O(n^2)$
  - Es inestable no mantiene el orden relativo a los registros

4. MOI. Shell (M. directo)

- ACCION metodShell(Vector x, ENTERO dx)
  - ENTERO i , j , k , salto ;
  - REAL aux
  - BOOL fin
  - salto ← dx ;
  - MIENTRAS (salto > 0)
    - salto = salto / 2
    - HACER
      - fin ← true
      - k ← n – salto
      - PARA i ← 0 HASTA (k-1)
        - j = i + salto
        - SI (x [i] < x[j])
          - aux ← v [i] ; v [i] ← v [j] ; v [j] ← aux
          - fin ← false
        - FIN_SI
        - i ← i + 1
      - FIN_)PARA
    - MIENTRAS(NO fin)
  - FIN_MIENTRAS
- FIN_ACCION

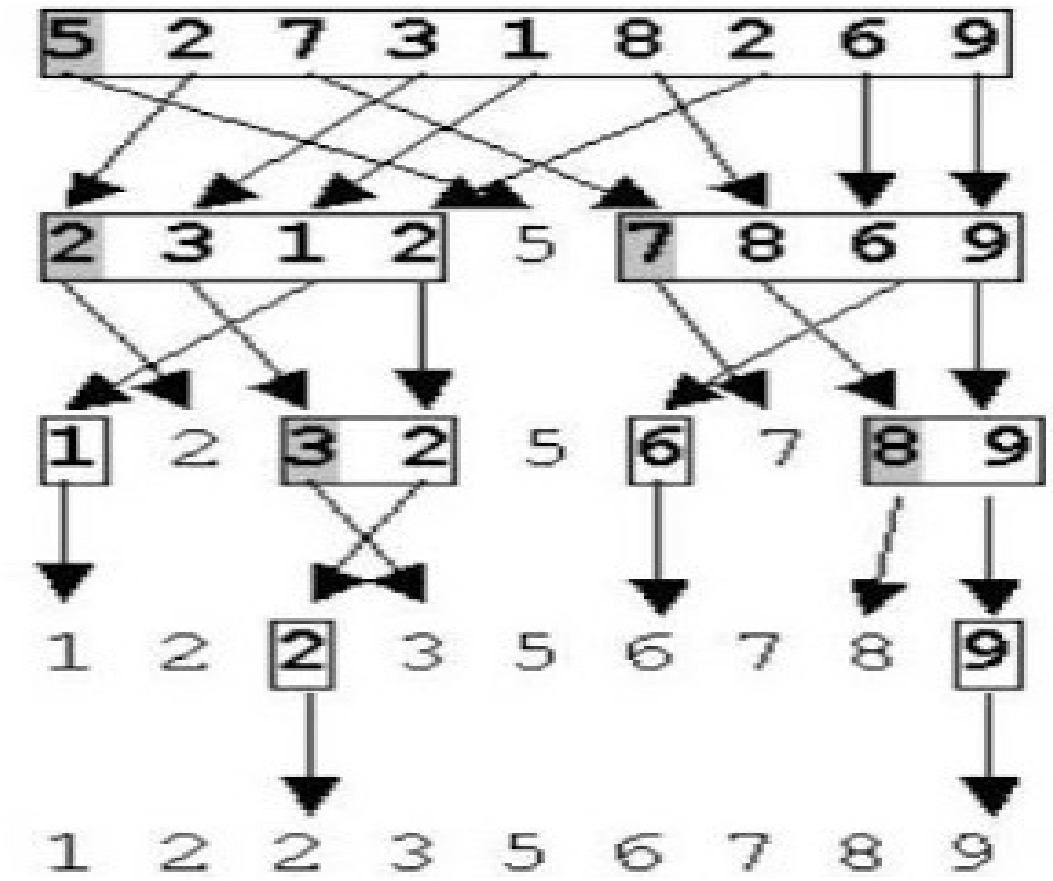
4. MOI. QuickSort (M. intercambio)

- Se basa en el principio de una mejora del método de *intercambio* y es denominado ordenamiento QuickSort por la velocidad conque ordena los elementos del vector. Fue creado por Charles Antony R. Hoare en 1960.

- **Los pasos son:**
  - Se toma el elemento ***dato*** de una posición cualquiera del vector.
  - Se trata de ubicar a ***dato*** en la posición correcta del vector, de tal forma que todos los elementos que se encuentran a su izquierda sean menores o iguales a ***dato*** y todos los elementos. Que se encuentran a su derecha sean mayores o iguales a ***dato.***
  - Se repite los pasos anteriores pero ahora para los conjuntos de datos que se encuentra a la izquierda y a la derecha de la posición correcta de ***dato*** en el vector.
  - El proceso termina cuando todos los elementos se encuentran en su posición correcta en el vector.

4. MOI. QuickSort (M. intercambio)

- Tomado de: http://fiec108.blogspot.com/



2. Ventajas

- **Ventajas**
  - Es el algoritmo mas rápido
  - En el mejor de los casos tiene un costo O(nlog(n)) ocurre cuando el pivote siempre queda en medio.
  - Uso de memoria(log(n))
- **Desventajas**
  - En el peor de los casos tiene un costo de $O(n^2)$ ocurre cuando el pivote siempre se inclina hacia un lado ósea genera un arreglo con un solo elemento y una segunda con el resto de elementos.

4. MOI. HeapSort (Montículo)

- Se basa en el principio de buscar el menor elemento del vector y colocarlo en la primera posición, luego se busca el segundo elemento mas pequeño y se coloca en la segunda posición y así sucesivamente.

- **Los pasos son:**
  - Seleccionar el menor elemento del vector de n elementos.
  - Intercambiar dicho elemento con el primero.
  - Repetir estas operaciones con los n-1 elementos restantes, seleccionando el segundo elemento, continuar con los n-2 elementos restantes hasta que solo quede el mayor.

4. Ventajas y desventajas

- **Ventajas**
  - Es el algoritmo no recursivo
  - En el mejor de los casos tiene un costo O(nlog(n)) ocurre cuando el pivote siempre queda en medio.
  - No utiliza memoria adicional. O(1)
- **Desventajas**
  - Algoritmo no estable, se comporta de manera ineficaz con daos del mismo valor, pueden cambiar el orden de los registros.
  - Método mas complejo

5. Métodos Ordenamiento Externo

- El ordenamiento externo es el ordenamiento de datos en memoria secundaria. Las grandes cantidades de datos están almacenadas en memoria secundaria y no podría hacerse en memoria principal.
  - El proceso de datos almacenados en varios archivos se denominan fusion o mezcla.

- **Clasificación:**
  - **Intercalación de archivos.** Es a fusión o unión de dos o mas archivos
  - **Ordenación de archivos**. Esta se lleva a cabo cuando el volumen de datos es demasiado grande y no posibles de ordenarlos en memoria principal. Los mas utilizados son:
    - **Mezcla directa.**
    - **Mezcla equilibrada**.

5. MOE: Intercalación

- El ordenamiento externo por intercalación se da en archivos ordenados por un determinado campo del registro.
  - Sean los files f1 y f2 debidamente ordenados por código, se pide obtener el archivo f3 también ordenado con los elementos de los f1 y f2

Solución:

f1

109	130	230	234

f2

105	121	134	138	240	243	250

f3

105	109	121	130	134	138	230	234	240	243	250

6. Resumen

- Siempre vamos a tener necesidad de hacer búsqueda de la información
- Siempre va haber necesidad de aplicar métodos de ordenamiento para optimizar los métodos de búsqueda.
- Cada uno de los métodos tiene sus propias características y utilizaremos cada una de ellas de acuerdo a cada necesidad.

Referencias

- Joyanes L. y Zahonero I. (2004) Algoritmos y estructura de datos. Una perspectiva en C. Madrid España: McGraw-Hill.
- Cortez A. (2013) Algoritmica técnicas algorítmicas. Lima. Peru: CEPREDIN
- Joyanes L. y Zahonero I. (2008) Estructura de datos en Java. Madrid, España: McGraw-Hill.
- Cairo O y Guardati S. (2006) Estructura de datos. Mexico: McHraw-Hill.
- Métodos de ordenamiento y su visualización. [Ultima actualizacion.09.agosto.2017] [Fecha de acceso 1935 horas de 14.junio.2020]. https://www.mql5.com/es/articles/3118#sel
- Algoritmos de ordenamiento. [Ultima actualización, no menciona] [Fecha de acceso 2010 horas de 14.junio.2020]. http://lwh.free.fr/pages/algo/tri/tri_es.htm
- **Popular Sorting Algorithms, https://randerson112358.medium.com/sorting-algorithms-6005e9ddd8c0**

Referencias

- Métodos de ordenamiento. https://es.wikipedia.org/wiki/Algoritmo_de_ordenamiento
- **Sorting algorithm. https://en.wikipedia.org/wiki/Sorting_algorithm**
- **3 Levels of Sorting Algorithms. https://www.youtube.com/watch?v=qk7b4-iyCJ4**
- **Sorting Algorithms Explained with Examples in Python, Java, and C++. https://www.freecodecamp.org/news/sorting-algorithms-explained-with-examples-in-python-java-and-c/**
- **Sorting algorithms compared. https://isaaccomputerscience.org/concepts/dsa_search_sorting_compared?topic=sorting**
- **Better Explained. Sorting Algorithms. https://betterexplained.com/articles/sorting-algorithms/**
- **5 Sorting Algorithms Every Programmer Should Know. https://betterprogramming.pub/5-basic-sorting-algorithms-you-must-know-9ef5b1f3949c**